Modelos Hierárquicos

5.1. Segundo o Compartilhamento de Memória

b) NUMA (*Non-Uniform Memory Access*)

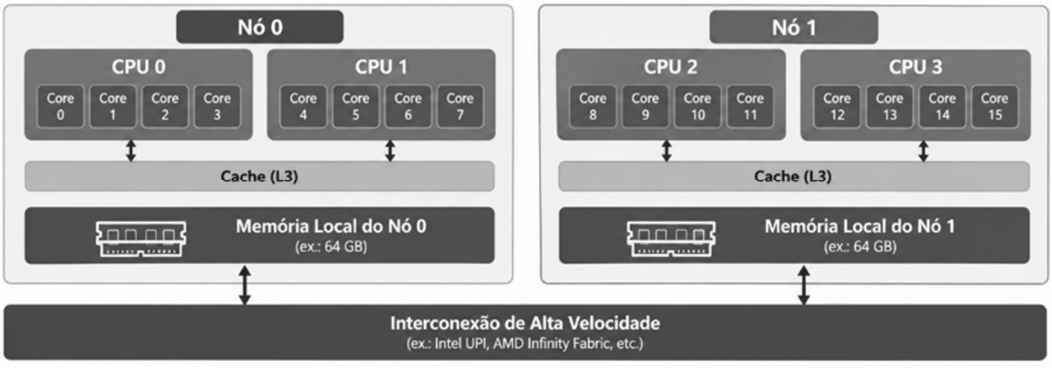
- A memória usada nessas máquinas é distribuída, implementada com múltiplos módulos que são associados um a cada processador.
- O espaço de endereçamento de um nó pode ser acessado por outro (não uniforme);
- Tempo de acesso local é menor (baixa latência) do que o tempo de acesso remoto (maior latência);
- O acesso entre os diferentes nós ocorre através de um barramento de interconexão de alta velocidade;

Modelos Hierárquicos

5.1. Segundo o Compartilhamento de Memória

b) NUMA (*Non-Uniform Memory Access*)

- A distância à memória não é sempre a mesma e depende do endereço desejado.
- Melhora o desempenho, pois não centraliza o acesso à uma única memória, utilizando um único barramento;

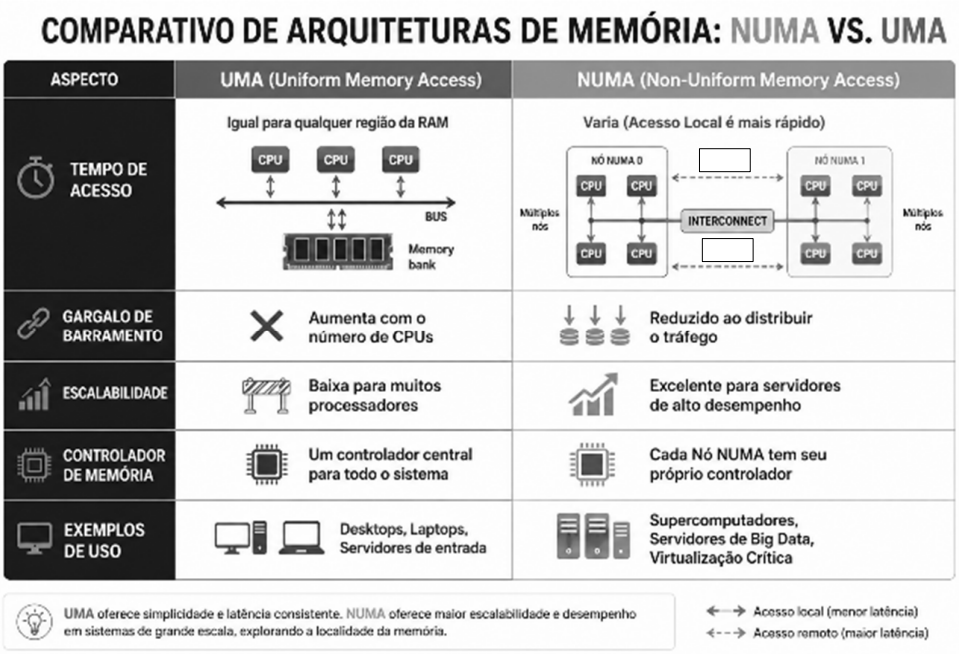


Muito utilizada em servidores modernos, em grandes ambientes de virtualização e Banco de Dados SQL.

Modelos Hierárquicos

5.1. Segundo o Compartilhamento de Memória

b) NUMA (*Non-Uniform Memory Access*)



Modelos Hierárquicos

5.1. Segundo o Compartilhamento de Memória

b) NUMA (*Non-Uniform Memory Access*)

- Essa classe pode ser subdividida em:
  - NCC-NUMA (*Non-Cache-Coherent Non-Uniform Memory Access*)
    - Variação de NUMA em que não há coerência de cache garantida em hardware;
  - CC-NUMA (*Cache-Coherent Non-Uniform Memory Access*)
    - Variação de NUMA em que há coerência de cache garantida em hardware;

Modelos Hierárquicos

5.1. Segundo o Compartilhamento de Memória

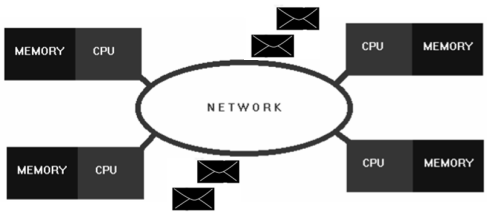
b) NUMA (*Non-Uniform Memory Access*)

- Essa classe pode ser subdividida em:

  - SC-NUMA (*Software-Coherent Non-Uniform Memory Access*)
    - A coerência não está implementada em hardware (como nas máquinas CC-NUMA), mas em software, de forma transparente ao usuário;
    - Essa camada de software é também conhecida como DSM (*Distributed Shared Memory*);
    - Só faz sentido em arquiteturas que não possuem coerência de cache em hardware.

Tipos de Comunicação

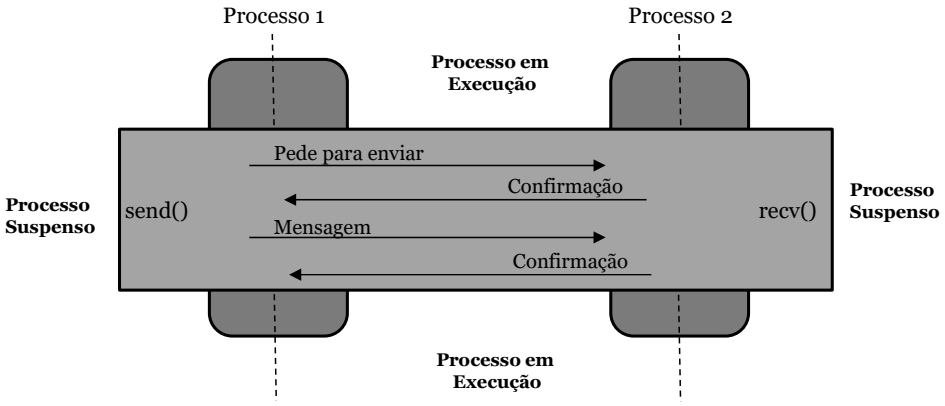
- Arquitetura com Memória Distribuída:
  - ✓ Neste tipo de paradigma de programação, a comunicação entre os diferentes fluxos de execução da aplicação é feita por meio do envio e recebimento de mensagens;



  - ✓ Há dois tipos de troca de mensagens: Síncrono e Assíncrono;
    - Modelo de Comunicação Síncrono.
      - ✓ Esse modelo de comunicação requer um protocolo de quatro fases para a sua implementação.

Tipos de Comunicação

- Arquitetura com Memória Distribuída:
  - Modelo de Comunicação Síncrono.
    1. Pedido de autorização para transmitir;
    2. Recebimento da autorização;
    3. Transmissão da mensagem;
    4. Recebimento da mensagem de reconhecimento;



Tipos de Comunicação

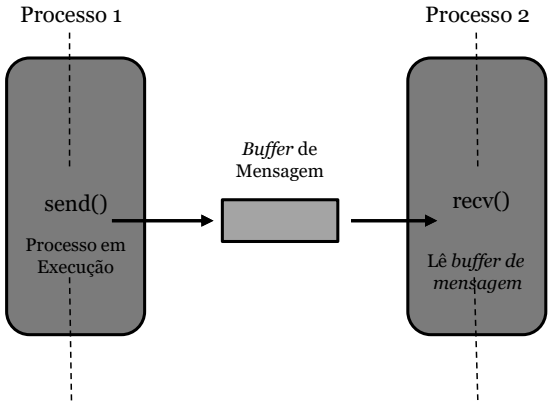
- Arquitetura com Memória Distribuída:
  - Modelo de Comunicação Síncrono.
    - ✓ Tanto o envio quanto a recepção de mensagens são bloqueantes;
      - A função de envio não retorna para o programa principal enquanto a mensagem não for completamente enviada;
      - A função de recepção também não retorna enquanto a mensagem não estiver disponível para o usuário no espaço de memória reservado na aplicação para isso;
    - ✓ Esse modo de comunicação é **simples**, **seguro** e **transparente** ao usuário;
    - ✓ Elimina a possibilidade de haver superposição entre processamento e transmissão das mensagens (reduzindo a possibilidade de exploração de paralelismo);

Tipos de Comunicação

- Arquitetura com Memória Distribuída:
  - Modelo de Comunicação Assíncrono.
    - ✓ A principal característica deste modelo de comunicação é a desvinculação do instante de tempo no qual ocorrem as operações de envio e recepção da mensagem;
      - Uso de áreas (*buffers*) destinadas ao armazenamento da mensagem enviada.
      - A operação de envio (*send*) se completa a partir da cópia de todo o conteúdo da mensagem do espaço de endereçamento do usuário para uma área de *buffer* da biblioteca ou do sistema operacional.

Tipos de Comunicação

- Arquitetura com Memória Distribuída:
  - Modelo de Comunicação Assíncrono.
    - ✓ Este modelo permite uma maior superposição entre o processamento da aplicação e a transmissão das mensagens, (aumentando as possibilidades de exploração de paralelismo);



Modelos Hierárquicos

- Arquitetura com Memória Distribuída:

  a) NORMA (*Non-Remote Memory Access*)

  - Como uma arquitetura tradicional inteira foi replicada na construção dessas máquinas, os registradores de endereçamento de cada nó, só conseguem endereçar a sua memória local.
  - Nesta classe, podem-se encontrar arquiteturas do tipo: MPP (*Massively Parallel Processors) e* COW (*Cluster of Workstations*);

Modelos Hierárquicos

- Arquitetura com Memória Distribuída:

  a) NORMA (*Non-Remote Memory Access*)

  a.1) MPP (*Massively Parallel Processors)* :

  - Supercomputadores - custam alguns milhões de dólares;
  - Usam processadores *multicores* em seus projetos;
  - nCube, CM5, Sun Ultra SPARC, IBM RS/6000, a família DEC Alpha, Cray T3E.
  - As MPPs também vêm equipadas com bibliotecas de software proprietárias (muito caras).
  - Sempre possuem *hardware* e *software* para monitoração do sistema, detecção de falhas e um excelente esquema de recuperação dessas falhas.

Modelos Hierárquicos

- Arquitetura com Memória Distribuída:
  a) NORMA (*Non-Remote Memory Access*)
    a.1) MPP (*Massively Parallel Processors)* :



Modelos Hierárquicos

- Arquitetura com Memória Distribuída:
  a) NORMA (*Non-Remote Memory Access*)
    a.2) COW (*Cluster of Workstations*);
    - É formado por PCs comuns, ligados por intermédio de uma rede local padrão (Ethernet);
    - Usam componentes comerciais que podem ser comprados e montados localmente;
    - Utilizam sistemas operacionais, softwares e bibliotecas livres como: Linux, OpenMP, OpenMPI, OpenACC, entre outras;

Modelos Hierárquicos

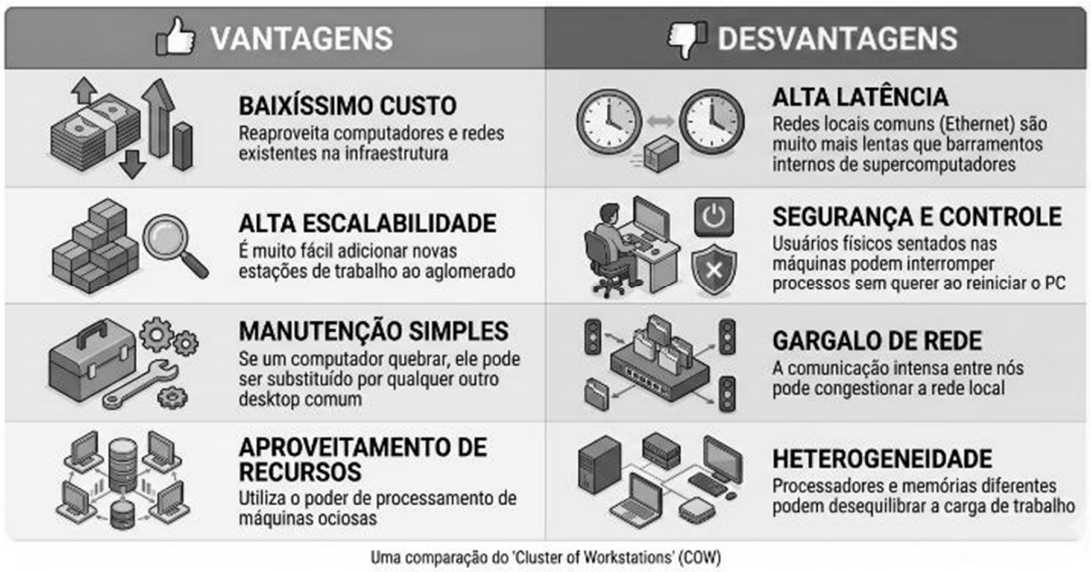
- Arquitetura com Memória Distribuída:
  a) NORMA (*Non-Remote Memory Access*)
    a.2) COW (*Cluster of Workstations*);





Modelos Hierárquicos

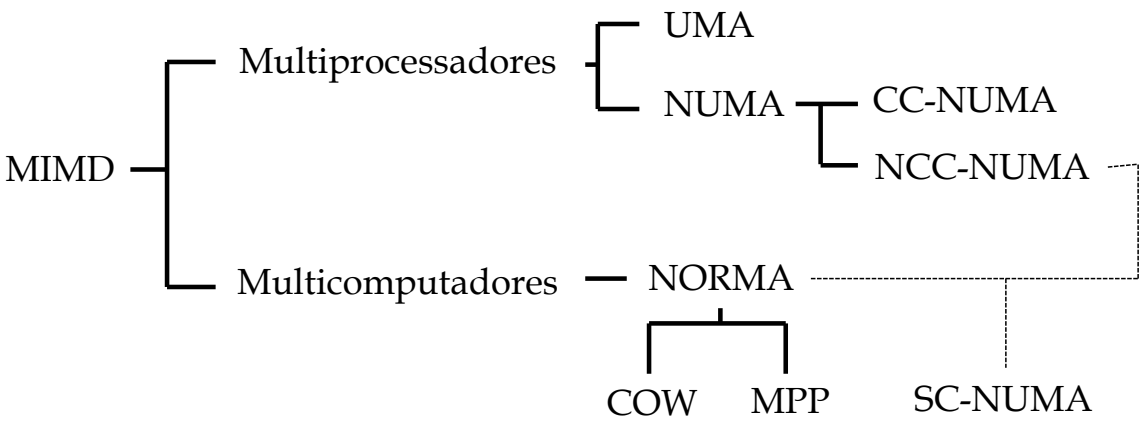
- Arquitetura com Memória Distribuída:
  a) NORMA (*Non-Remote Memory Access*)
    a.2) COW (*Cluster of Workstations*);



Uma comparação do 'Cluster of Workstations' (COW)

Modelos Hierárquicos

5.1. Segundo o Compartilhamento de Memória

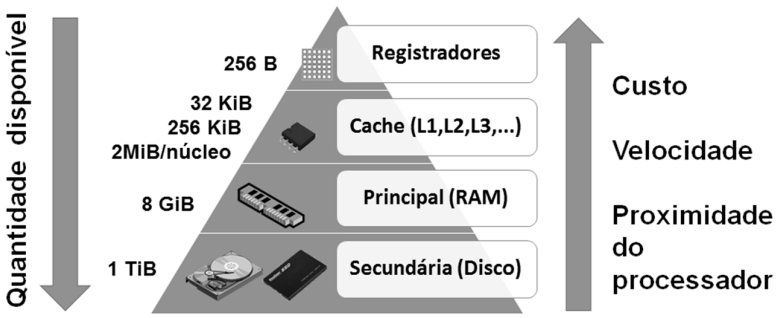


Sumário

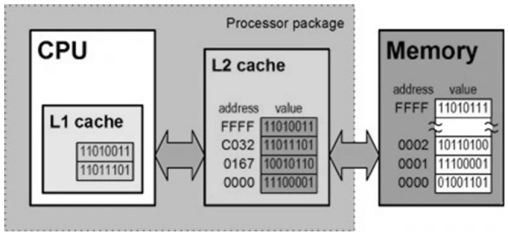
1. Motivação e Objetivos
2. Introdução
3. Classificações e Terminologias
    3.1. Classificação de *Flynn*
    3.2. Classificação de Duncan
4. Tipos de Comunicação
    4.1. Arquitetura com Memória Compartilhada
        a) UMA (*Uniform Memory Access*)
        b) NUMA (*Non-Uniform Memory Access*)
    4.2. Arquitetura com Memória Distribuída
        4.2.1. Síncrono
        4.2.2. Assíncrono
        4.2.3. NORMA (*Non-Remote Memory Access*)
5. Coerência de Cache
    5.1. Outras Classificações
    5.2. Aceleradores
6. Conclusão
7. Referências

Coerência de Cache

- A grande diferença de velocidade entre processadores e memória, vem representando um grande desafio para os projetistas de arquiteturas de computadores;
  - ✓ Pouco adianta, o processador cada vez mais rápido, se não puder ser alimentado pelo subsistema de E/S com dados na mesma velocidade: **Afeta o desempenho global do sistema**;
- Uma alternativa para atenuar essa diferença é a utilização de uma hierarquia de memória;



Coerência de Cache



- Um endereço requisitado é procurado no primeiro nível da hierarquia, e caso não encontrado, é passado para o nível inferior (de forma recursiva);

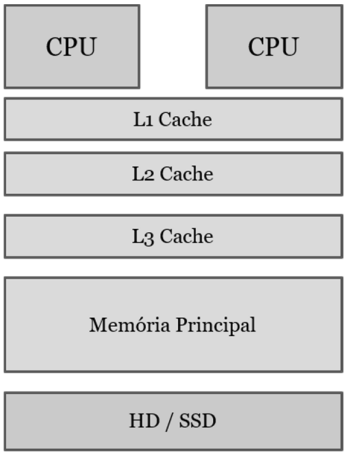


Figura 1

- Arquitetura boa para poucos processadores (ganho obtido é limitado):
  - ✓ Latência da rede de interconexão (processador-cache);
  - ✓ Conflitos na cache:
    - Cada processador vai tentar manter os seus blocos mais usados na cache, tendo que para isso, dependendo do tamanho da cache, substituir os blocos dos outros processadores;

Coerência de Cache

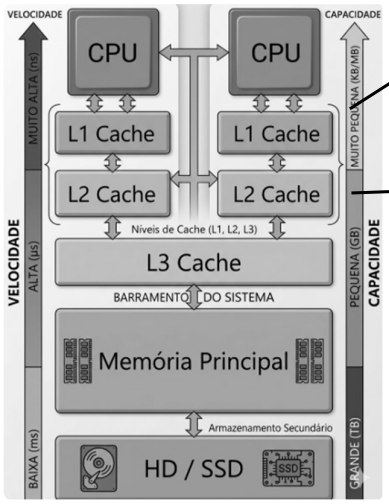


Figura 2

A presença de caches privadas em multiprocessadores introduz problemas de **coerência de cache**.

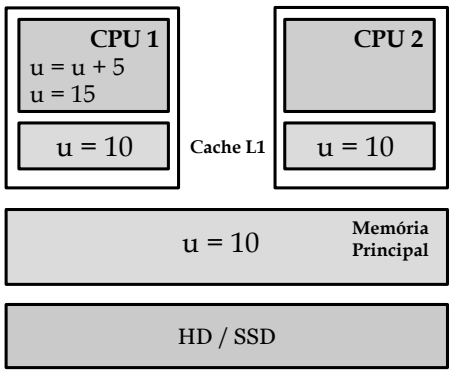
Cada processador pode atualizar a sua cópia local, não se preocupando com a existência de outras cópias.

- Não permitir que dados compartilhados para operações de escrita sejam colocados em caches (**instruções e dados privados**);
- Compilador fica responsável pela colocação de *tag*;

- Quando dois processos compartilham a mesma memória através de caches diferentes, existe o risco de uma posição de memória não possuir mais o valor mais recente desse dado;
- Isso é o que chamamos de **Inconsistência de Cache (ou de Dados)**.

Coerência de Cache

- Os tipos mais comuns de Inconsistência de Cache podem ser:
  1. Inconsistência no Compartilhamento de Dados.

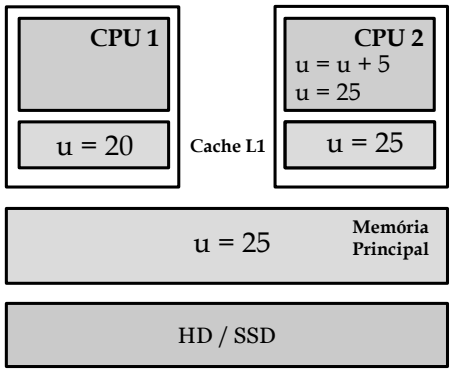


- As CPUs possuem caches privadas e estão acessando uma memória principal (MP) compartilhada;
- Operações de leitura da MP podem ser efetuadas sem problemas e/ou prejuízos;
- Operações de escrita, em variáveis que estão compartilhadas na cache, podem gerar inconsistência de dados;

Coerência de Cache

- Os tipos mais comuns de Inconsistência de Cache podem ser:

  2. Inconsistência na Migração de Processos.

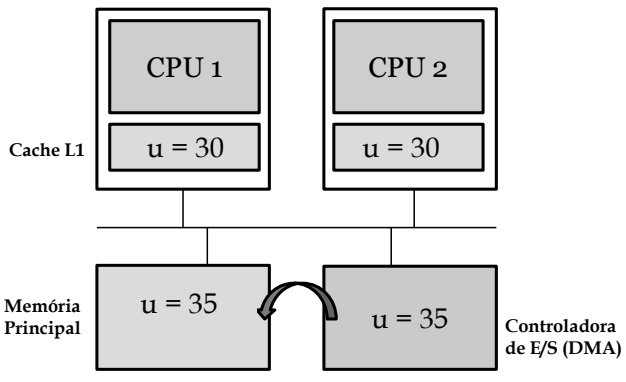


- Fenômeno *Pack Rat*:
  - ✓ Processo deixa de executar em um processador por uma operação de E/S;
  - ✓ Pode voltar a ser executado em outro processador;

Coerência de Cache

- Os tipos mais comuns de Inconsistência de Cache podem ser:

  3. Inconsistência de E/S.



- Arquitetura tradicional o processador era responsável por operações de E/S;
- Objetivo de sobrepor operações de E/S e processamento, as unidades de E/S ganharam em autonomia;
- Execução da operação de E/S e pelo armazenamento ou leitura de dados diretamente da memória principal (DMA);